D 28 M 09 A 26

«Modelo de programación funcional»

Entendemos que la programación funcional es un paradigma de programación que se enfoca en el uso de funciones matemáticas puras para resolver problemas de programación.

En lugar de enfatizar en el estado y cambios de un programa, se centra en el cálculo y transformación de datos.

Las funciones puras son aquellas que siempre devuelven el mismo resultado dado el mismo input y no tienen efectos secundarios en el sistema, lo que hace que los programas sean más predecibles y fáciles de probar.

Hay varios lenguajes de programación que se basan en el paradigma de la programación funcional, como Lisp, Haskell y Scala, así como lenguajes más populares que incluyen características funcionales como JavaScript y Python.

La programación funcional es un paradigma declarativo. Nos enfocaremos en "qué" estamos haciendo y no en "cómo" se está haciendo que sería el enfoque imperativo. Esto quiere decir que nosotros expresamos nuestra lógica sin describir controles de flujo, no usaremos ciclos o condicionales.

Ejemplo en Java

Java, es un lenguaje orientado a objetos, sin embargo en versiones recientes nosotros podemos hacer uso de la programación funcional.

En este ejemplo, la problemática es conocer la cantidad de elementos en la lista mayores a 10.

D 28 M 09 A 26

```
List<Integer> numeros = List.of(10, 6, 4, 15, 55, 78, 12, 9, 8);
```

Nosotros podemos resolver la problemática utilizando ciclos y condicionales. Este sería el enfoque imperativo.

```
// Declarativo
List<Integer> numeros = List.of(10, 6, 4, 15, 55, 78, 12, 9, 8);
int contador = 0;
for(int numero : numeros) {
    if(numero > 10) {
        contador ++;
    }
}
System.out.println(contador);
```

El enfoque imperativo sería el siguiente.

```
// Imperativo
Long result = numeros.stream().filter(num -> num > 10).count();
System.out.println(result);
```

Como observamos, en el enfoque imperativo las líneas de código se reducen y nuestro código se toma más legible.

D 28 M 09 A 26

Entendemos por programación funcional un lenguaje de programación declarativo donde el programador especifica lo que quiere hacer, en lugar de lidiar con el estado de los objetos. Es decir, las funciones estarían en un primer lugar y nos centraremos en expresiones que pueden ser asignadas a cualquier variable.
El origen del modelo de programación funcional pese a ser algo de relativa reciente aceptación, tiene su origen en el cálculo lambda. El cálculo lambda es un sistema desarrollado en la década de los 30 del siglo XX, donde buscaban investigar la naturaleza de las funciones y la computabilidad.

En el lenguaje de programación funcional, contamos con características clave que lo diferencian de otros paradigmas de programación:
- No efectos colaterales: Una función, si tiene todos los parámetros definidos por valor y no se hacen asignaciones a variables globales, no tendrá efectos colaterales
- Transparencia referencial: no depende de nada más que de los valores de sus subexpresiones, si las tuviera.
- Tipos de datos limpios: Es un lenguaje declarativo que produce siempre el mismo resultado. El almacenamiento de datos se gestiona automáticamente y las funciones son valores de primera clase, por lo que pueden pasarse como argumentos, usarse en expresiones y almacenarse en estructuras de datos.
- Composición: La programación funcional se enfoca en la composición de pequeñas funciones para generar funciones más complejas. Esto permite originar código más legible y fácil de probar.

D 28 | M 09 | A 26

Lenguajes de programación funcional que existen:

En la actualidad contamos con un buen puñado de lenguajes funcionales y paradigmas de programación que los aceptan:

- LISP
- ML
- Haskell
- OCaml
- F#
- Erlang
- Clojure
- Scala

Además, existen muchos lenguajes de programación conocidos con los que podremos aplicar modelos de programación funcional entre sus paradigmas.

- Perl
- Ruby
- Visual Basic. NET
- Dylan
- Javascript
- Python

« VENTAJAS E INCONVENIENTES »

- Los programas no tienen un estado definido.
- Son ideales para la paralelización
- El código se puede verificar con facilidad
- Se trata de un código verificable, sobre todo para funciones sin estado se puede verificar
- Código más preciso y corto.
- Los datos variables no se pueden modificar
- No se puede acceder a grandes cantidades de datos
- No es recomendable para conexiones a base de datos y servidores
- No es adecuado para muchas recursiones de la misma pila
- Con la programación recurrente, puede dar pie a errores graves.

Scanned with CamScanner

D 28 M 09 A 26

La programación funcional nos permitirá desarrollar software mucho más legible y fácil de testear, nos centramos en qué estamos haciendo y no en cómo se esta haciendo.

Scanned with CamScanner

Cálculo Lambda de Alonzo Church

D 28 | M 04 | A 26

El cálculo lambda de Alonzo Church es un modelo matemático de la computación que utiliza funciones para representar y resolver problemas. Se considera una de las bases de los lenguajes de programación funcionales, ya que permite expresar cálculos mediante la aplicación y combinación de funciones.

Lo más importante del cálculo lambda de Alonzo Church es que demostró que en cualquier cálculo computable puede representarse mediante funciones, convirtiéndose en una de las bases teóricas de la programación funcional y de la informática moderna.

Utilica una notación parecida a una fórmula para representar funciones.

La forma básica es:

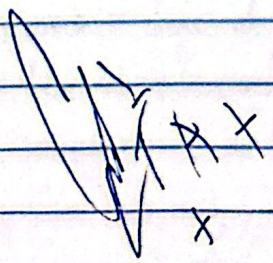
$\lambda x.\ x + 1$

Se lee: "una función que recibe x y devuelve x + 1".

Lo importante es que λ indica que estamos definiendo una función.

Las expresiones o términos lambda, se escriben utilizando 3 elementos básicos: variables, funciones y aplicaciones

Variable	<nombre>	x	una variable llamada "x"
Función	λ<parámetro>. <término>	λx.x	una función con parámetro "x" y cuerpo "x"
Aplicación	<término> <término>	(λx.x)a	"λx.x" con el parámetro "a"



D 28 | M 09 | A 26

- Variables: representan valores o datos que utilizan dentro de una función.
  Ejemplo: x, y, z
- Abstracciones: permiten definir funciones utilizando el símbolo λ
  Ejemplo: λx. x + 2
- Aplicaciones: consisten en ejecutar una función sobre un valor o argumento.
  Ejemplo: (λx. x + 2) (3) cuyo resultado es 5.

Las variables representan datos, las abstracciones definen funciones y las aplicaciones permiten usar funciones para realizar cálculos.

Scanned with CamScanner

• Elixir •

Elixir es un lenguaje de programación funcional creado por José Valim y presentado en 2011. Está construido sobre la máquina virtual de Erlang (BEAM), por lo que puede aprovechar muchas de las características de Erlang.

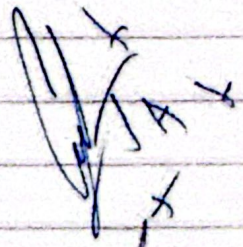
Elixir busca proporcionar una sintaxis más moderna y sencilla, manteniendo las capacidades de Erlang para crear sistemas concurrentes, distribuidos y resistentes a errores.

Es un lenguaje funcional

- Se ejecuta sobre la máquina virtual BEAM
- Permite manejar muchas tareas simultáneamente
- Tiene tolerancia a fallos
- Su sintaxis es relativamente sencilla y flexible
- Permite desarrollar servidores y aplicaciones web
- Puede utilizar bibliotecas y herramientas del ecosistema de Erlang.

Elixir se utiliza principalmente en:

- Aplicaciones web
- Sistemas distribuidos
- Servicios en tiempo real
- Plataformas de comunicación
- Servidores con muchos usuarios conectados
- Sistemas que requieren alta disponibilidad.



Scanned with CamScanner

Erlang resultó adecuado porque permite crear sistemas capaces de manejar muchas conexiones simultáneas, algo fundamental para una aplicación de mensajería.

¿En qué parte de WhatsApp se utilizaba Erlang?

Se utilizó principalmente en el lado del servidor, no como lenguaje utilizado para crear toda la aplicación que vemos en el teléfono.

De manera simplificada, cuando una persona envía un mensaje:

Teléfono → Servidores de WhatsApp → Servidores de WhatsApp → Teléfono del destinatario

WhatsApp utilizó a Erlang por:

- ✓ Concurrencia
- ✓ Tolerancia a fallos
- ✓ Alta disponibilidad
- ✓ Escalabilidad
- ✓ Comunicación en tiempo real

¿Elixir, qué relación tiene con WhatsApp?

Aquí es importante hacer una diferencia:

WhatsApp es conocido principalmente por el uso de Erlang en su infraestructura de servidores. No significa que WhatsApp esté desarrollado completamente en Elixir.

Elixir se relaciona con WhatsApp porque funciona sobre la maquina virtual BEAM, la misma tecnología utilizada por Erlang, lenguaje que WhatsApp emplea en su infraestructura de servidores; se relaciona porque comparte con Erlang la máquina virtual y las características de concurrencia y tolerancia a fallos que hacen posible manejar sistemas de mensajería a gran escala.

Scanned with CamScanner

Evelin

D M A

Scribe

Funciones de orden superior: map, filter y fold/reduce sobre listas

Las funciones de orden superior son funciones que reciben otras funciones como argumentos o las utilizan para trabajar con datos. En programación funcional, como Haskell, se usan para procesar listas de manera más sencilla, sin necesidad de escribir tantos ciclos.

1. Función map

La función map sirve para aplicar una operación a cada elemento de una lista y devolver una nueva lista con los resultados.

Ejemplo en Haskell:

haskell

```
map (*2) [1,2,3,4]
```

Resultado

haskell

```
[2,4,6,8]
```

¿Cómo funciona?

- Toma el número 1 y lo multiplicas por 2.
- Toma el número 2 y lo multiplicas por 2.
- Repite el proceso con cada elemento.
- Devuelve una lista con todos los resultados.

Otro ejemplo

haskell

```
map (+5) [1,2,3]
```

Resultado: [6,7,8]

Scanned with CamScanner

2. Función filter

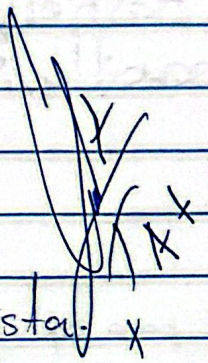
La función filter permite seleccionar únicamente los elementos de una lista que cumplen una condición.

Ejemplos en Haskell:

```
haskell
filter even [1,2,3,4,5,6]
```

Resultado

```
haskell [2,4,6]
```



¿Como funciona?

- Revisa cada número de la lista.
- Comprueba si es par mediante even
- Conserva los numeros pares y descarta los impares.

Otro ejemplo

```
haskell
filter (>3) [1,2,3,4,5,6]
```

Resultado: [4,5,6]

3. Función fold o reduce

La función fold sirve para combinar los elementos de una lista y obtener un resultado final, por ejemplo, una suma o un producto.

En Haskell existen principalmente foldl y foldr

Ejemplo con foldl:

```
haskell
foldl (+) 0 [1,2,3,4]
```

Resultado

```
haskell
[1,0]
```

Scanned with CamScanner

¿Como funciona

- Comienza con el valor inicial 0.
- Suma el primer elemento: 0+1=1.
- Después suma el segundo: 1+2=3
- Continua hasta terminar la lista
- El resultado final es 10

Otro ejemplo

haskell
foldl (*) 1 [1,2,3,4]

Resultado: 24

Scanned with CamScanner

D 19 M 19 A 26

Apunte

```
Count:=0
total=0
repetir
    Count:= count+1
    total= total+count

Until count = (n)
```

$$\sum_{i=1}^{n=5}$$

$1+2+3+4+5 = 15$

Sum:

[ ]

Sum [1 . . 5]

sum [15]

« Notación infija, prefija y postfija »

D | M | A

Scribe

Característica	Notación infija	Notación prefija	Notación postfija
Definición	El operador se coloca entre los operandos.	El operador se coloca antes de los operandos.	El operador se coloca después de los operandos.
Posición del operador	Entre los valores	Al inicio de la expresión	Al final de la exp.
Ejemplo	5 + 3	+ 5 3	5 3 +
Representación en Haskell	5 + 3	(+) 5 3 div + 5	5 3 + (representación postfija, no expresión directa).
Uso de paréntesis	Puede necesitar paréntesis para indicar el orden de las operaciones.	Generalmente no necesita paréntesis si se conoce la estructura.	Generalmente no necesita paréntesis si se conoce la estructura
Evaluación	Considera la prioridad y asociatividad de los operadores.	Se identifica primero el operador y después sus operandos.	Se utilizan los operandos antes de aplicar el operador
Fácil comprensión	Es fácil de entender porque es la forma común de las operaciones matemáticas.	Puede ser menos familiar para las personas.	Puede ser más difícil de interpretar visualmente.
Ventaja	Permite escribir operaciones de manera natural.	Facilita la representación de operaciones sin depender de la prioridad tradicional	Facilita la evaluación mediante una pila de valores
Desventaja	Puede generar confusión cuando hay operadores.	Puede resultar difícil de leer para principiantes.	Puede resultar poco intuitiva.
Importancia	Es importante porque permite escribir expresiones matemáticas de manera sencilla y facilita la lectura del código en Haskell	Es importante porque ayuda a comprender cómo se organizan las operaciones y cómo se pueden representar sin utilizar la profundidad tradicional	Es importante porque permite evaluar expresiones mediante algoritmos y ayuda a comprender el funcionamiento.
Resultado	8	8	8

Scanned with CamScanner

« 2.3 Intervalos »

Los intervalos en Haskell, tambien conocidos como Ranges, son una forma sencilla de crear listas de valores consecutivos o que sigoen una secuencia determinada, sin necesidad de escribir cada elemento manualmente.

Se utilizan principalmente para generar números, letras y otras secuencias.

Sintaxis básica:

[inicio..fin]

Tipos de intervalos

- Intervalos ascendentes

Generan números de menor a mayor.

[1..10]

Resultado

[1,2,3,4,5,6,7,8,9,10]

- Intervalos con incrementos

Permite avanzar de una cantidad determinada

[2,4..12]

Resultado

[2,4,6,8,10,12]

- Intervalos descendentes

Se utilizan para generar secuencias de mayor a menor, indicando los dos primeros valores.

[10,9..1]

Resultado

[10,9,8,7,6,5,4,3,2,1]

ideas HERNANDEZ JIORGANA

Scanned with CamScanner

- Intervalos de caracteres

También permite generar secuencias de letras

['a'..'f']

Resultado

"abcdef"

- Intervalos infinitos

Son secuencias que no tienen un límite final

[1..]

Para mostrar solamente los primeros cinco números:

take 5 [1..]

Resultado

[1,2,3,4,5]

Sirven para:

- Crear listas automáticamente
- Generar números pares e impares
- Realizar conteos
- Trabajar con secuencias de caracteres
- Facilitar operaciones matemáticas.

Scanned with CamScanner

« 2.4 Operadores »

Los operadores son símbolos o palabras que permiten realizar diferentes operaciones con los datos de un programa. En Haskell se utilizan para realizar cálculos matemáticos, comparar valores, combinar condiciones y trabajar con listas.

Son importantes porque permiten expresar instrucciones de manera sencilla y realizar operaciones entre diferentes tipos de datos.

Tipos de operadores

1. Operadores aritméticos

Se utilizan para realizar operaciones matemáticas

Operador	Función	Ejemplo
+	Suma	5 + 3 = 8
-	Resta	8 - 3 = 5
*	Multiplicación	5 * 3 = 15
^	Potencia	2 ^ 3 = 8
div	División entera	10 div 3 = 3
mod	Residuo	10 mod 3 = 1

2. Operadores relacionales

Permiten comparar dos valores y determinar si una condición es verdadera o falsa.

Operador	Significado
==	Igual que
/=	Diferente de
>	Mayor que
<	Menor que
>=	Mayor o igual
<=	Menor o igual

ideas
HERNÁNDEZ JIORGANA

Scanned with CamScanner

3. Operadores lógicos

Se utilizan para combinar condiciones.

AND

OR

NOT

4. Operadores para listas

Haskell cuenta con operadores especiales para trabajar con listas.

: Agrega al elemento al inicio de una lista.

++ Une dos listas

!! Obtiene un elemento por su posición.

Los operadores permiten realizar cálculos, tomar decisiones mediante condiciones, comparar información y manipular listas. Son fundamentales para construir expresiones y desarrollar programas en Haskell.

Scanned with CamScanner

« 2.7 Evaluación perezosa »

La evaluación perezosa (Lazy Evaluation) es una característica muy importante en Haskell que consiste en no realizar un cálculo hasta que realmente se necesita su resultado.
En otras palabras, Haskell no ejecuta todas las operaciones inmediatamente, sino que espera hasta que algún resultado sea solicitado. Esto permite ahorrar recursos y trabajar con listas o secuencias muy grandes, incluso infinitas.

En la programación tradicional, muchas operaciones se realizan inmediante. En cambio, Haskell utiliza la evaluación perezosa para esperar hasta que el valor sea necesario.

Por ejemplo:

```
numeros = [1..]
take 5 numeros
[1,2,3,4,5]
```

Esto sucede porque solamente se solicitan los primeros cinco elementos

Ventajas

- Evita cálculos innecesarios
- Permite trabajar con listas infinitas
- Puede mejorar el aprovechamiento de memoria
- Facilita la creación de programas más flexibles
- Permite procesar información conforme se necesita.

Scanned with CamScanner